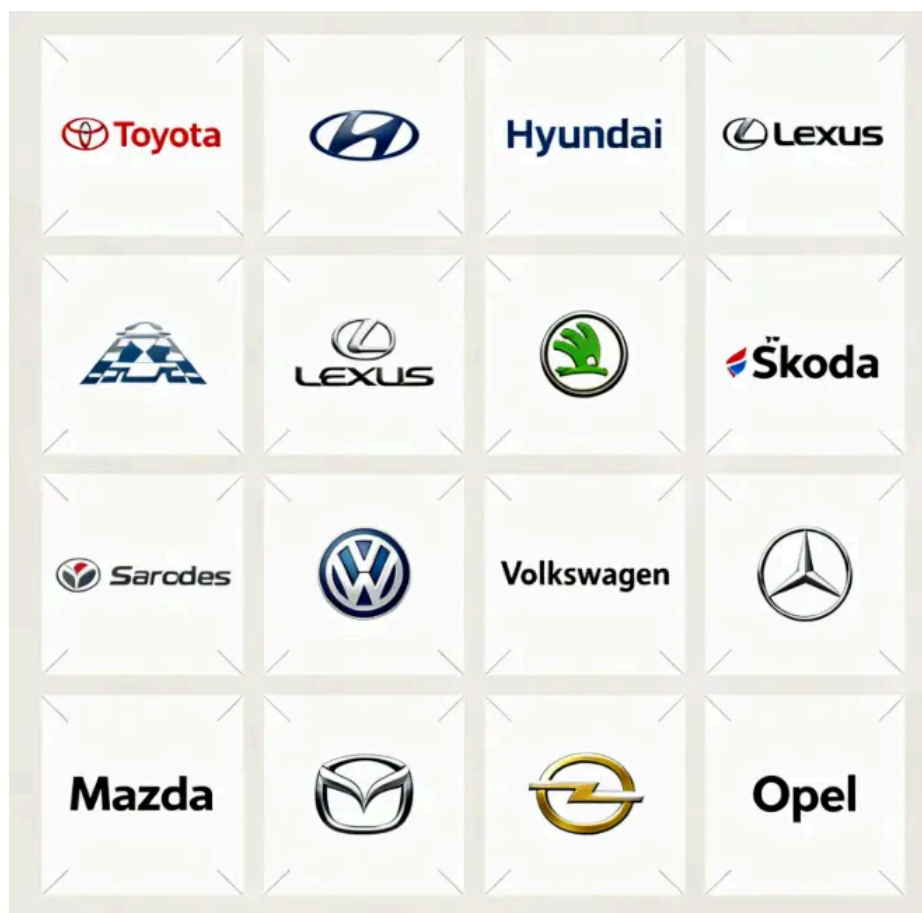


---

# WorkShop 2 Part 2

## Neural Networks



Diego Fiz García

# Table of Contents

---

## 1. Introduction

1.1. Background and Motivation

1.2. Research Objectives

## 2. Methodology

2.1. Dataset and Preprocessing

2.2. Model Architectures

2.3. Training Configuration

## 3. Results and Analysis

3.1. Quantitative Performance Comparison

3.2. Analysis of Training Dynamics

## 4. Discussion

4.1. The Decisive Impact of Skip Connections

4.2. Limitations and Future Work

## 5. Conclusion

## References

## Python Code

# 1. Introduction

---

## 1.1. Background and Motivation

The automated classification of car brand logos presents a nuanced computer vision challenge. Many logos exhibit significant visual similarities, such as circular enclosures and geometric motifs, demanding fine-grained discriminative capabilities. Moreover, real-world images introduce variability in scale, lighting, and occlusion. While deep CNNs are the standard for such tasks, their performance is known to degrade as depth increases—a phenomenon not caused by overfitting but by optimization challenges. This degradation is primarily attributed to the vanishing gradient problem, where gradients diminish exponentially during backpropagation, impeding learning in earlier layers.

## 1.2. Research Objectives

This study was designed to empirically investigate the practical impact of this theoretical problem. The primary objectives were:

1. To quantify the severity of the training degradation problem in a deep, conventional CNN applied to logo classification.
2. To determine if residual (skip) connections can effectively overcome this degradation.
3. To comparatively analyze the quantitative impact of skip connections on model accuracy, convergence, and parameter efficiency.

# 2. Methodology

---

## 2.1. Dataset and Preprocessing

The study utilized the "Car Brand Logos" dataset from Kaggle, which contains 2,513 images across eight brands. The dataset was partitioned using stratified sampling into training (70%), validation (15%), and test (15%) sets. All images were standardized to a 128×128 pixel resolution and normalized. To enhance generalization, data augmentation—including random rotations, shifts, and zooms—was applied to the training set.

## 2.2. Model Architectures

Two models were developed for comparison. The **Baseline CNN** followed a conventional sequential design with three convolutional blocks and a dense classification head, totaling 8.71 million parameters. The **ResNet Architecture** implemented residual learning via skip connections.

It comprised an initial block and seven residual blocks in three stages, computing  $y = F(x) + x$ . This design, with only 3.20 million parameters, was significantly more efficient.

### 2.3. Training Configuration

Both models were trained for up to 80 epochs using the Adam optimizer, a batch size of 32, and categorical cross-entropy loss. A `ReduceLROnPlateau` scheduler adjusted the learning rate, and early stopping prevented overfitting.

## 3. Results and Analysis

### 3.1. Quantitative Performance Comparison

The performance disparity between the two architectures was stark. The baseline model failed to learn, achieving a final validation accuracy of just 13.72%, barely above random chance (12.5%). In contrast, the ResNet model demonstrated successful learning, reaching 58.84% validation accuracy. This represents a 4.3-fold improvement while using 63.3% fewer parameters. Figure 1 shows that ResNet is superior across all metrics, including a 50.95% lower validation loss.

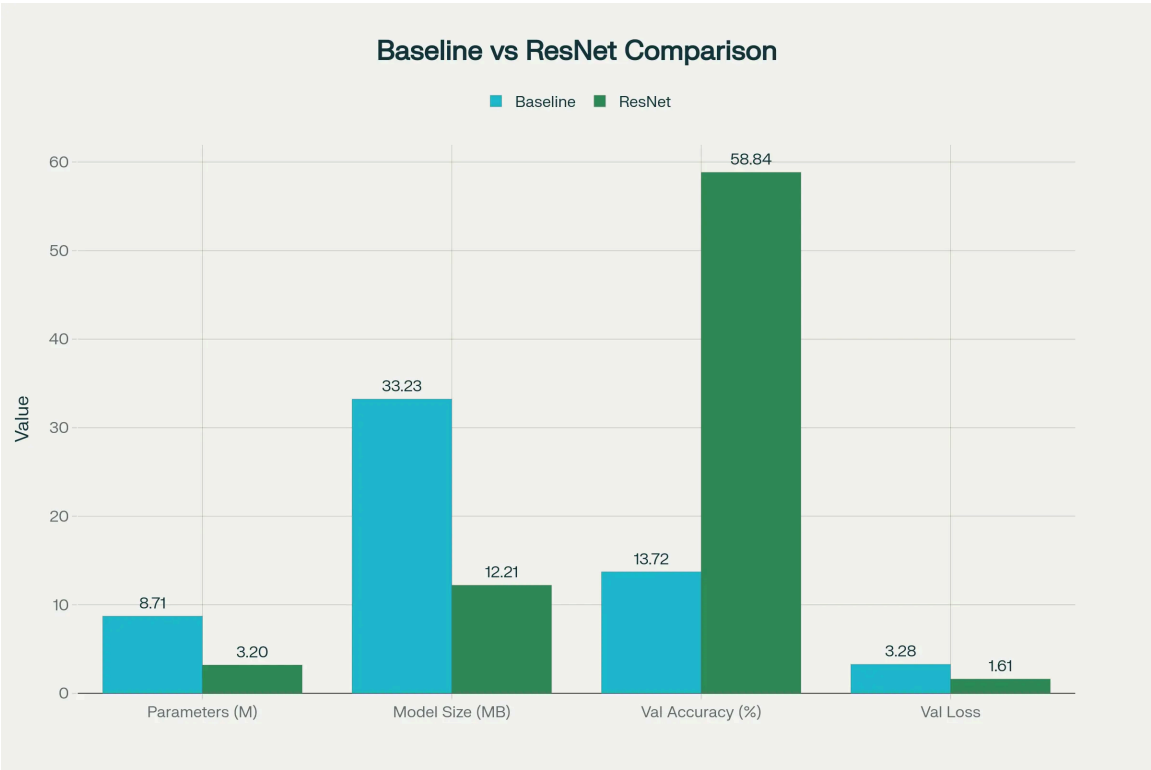


Figure 1: Comprehensive performance and efficiency comparison between the Baseline CNN and ResNet.

### 3.2. Analysis of Training Dynamics

The training curves provide a clear diagnostic. The baseline model's validation accuracy (Figure 2) remained stagnant, a classic symptom of the vanishing gradient problem. Conversely, the ResNet model's accuracy shows a healthy upward trend, rising from 14% to a peak of 58.84%. This demonstrates that skip connections established an unimpeded "gradient highway," allowing the network to learn effectively and rapidly, as detailed in Table 1.

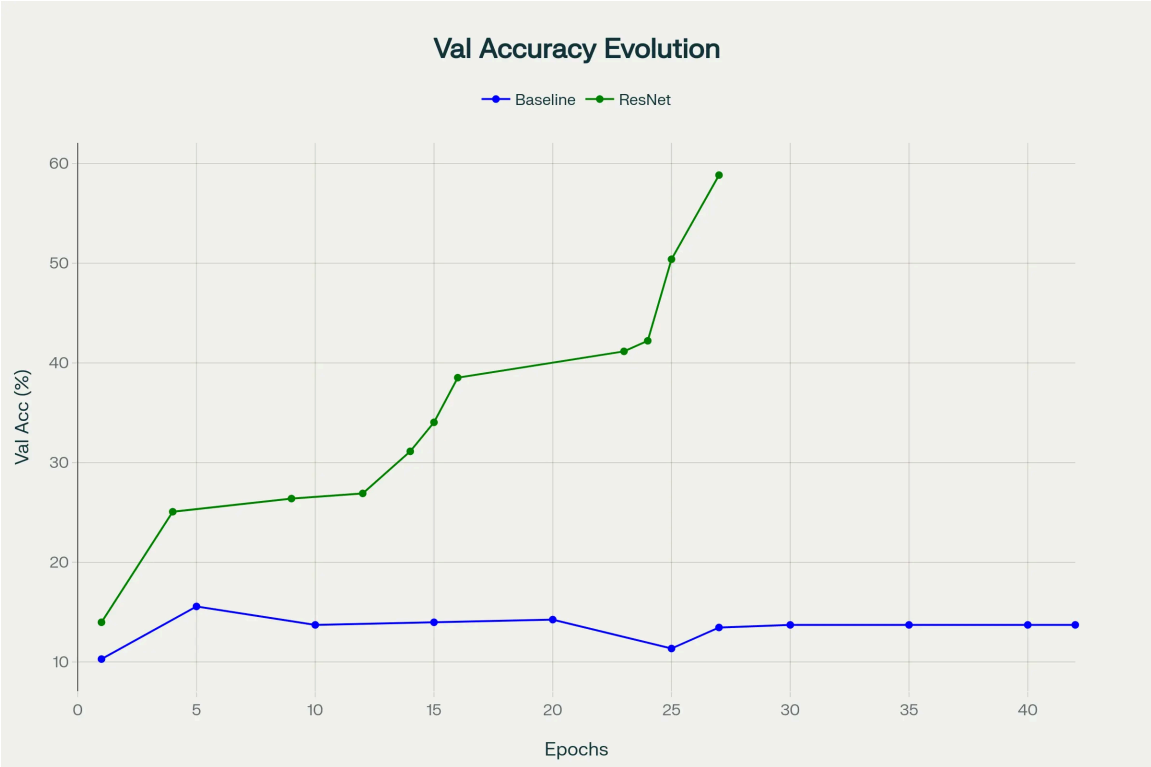


Figure 2: Validation accuracy evolution during training. ResNet exhibits steady improvement, while the Baseline CNN stagnates.

Table 1: Convergence speed analysis.

| Epoch | Baseline Val Acc | ResNet Val Acc | ResNet Advantage |
|-------|------------------|----------------|------------------|
| 4     | 12.1%            | 25.1%          | +13.0 pp         |
| 9     | 14.3%            | 26.4%          | +12.1 pp         |
| 15    | 14.0%            | 34.0%          | +20.0 pp         |
| 27    | 13.5%            | 58.8%          | +45.3 pp         |

## 4. Discussion

### 4.1. The Decisive Impact of Skip Connections

The results provide compelling evidence for the transformative impact of skip connections. Their primary advantage is mitigating vanishing gradients. By providing an identity path for backpropagation, they ensure gradients flow to earlier layers without exponential decay. The mathematical formulation of a residual block,  $H(x) = F(x) + x$ , guarantees that the gradient includes a direct term, preventing the optimization from collapsing.

This innovation leads to superior parameter efficiency. The ResNet achieved 4.3 times better accuracy with a 63.3% smaller model (Figure 3), proving that intelligent architecture is more critical than parameter count. The stability and speed of convergence further underscore this, translating to more reliable development and reduced computational expense.

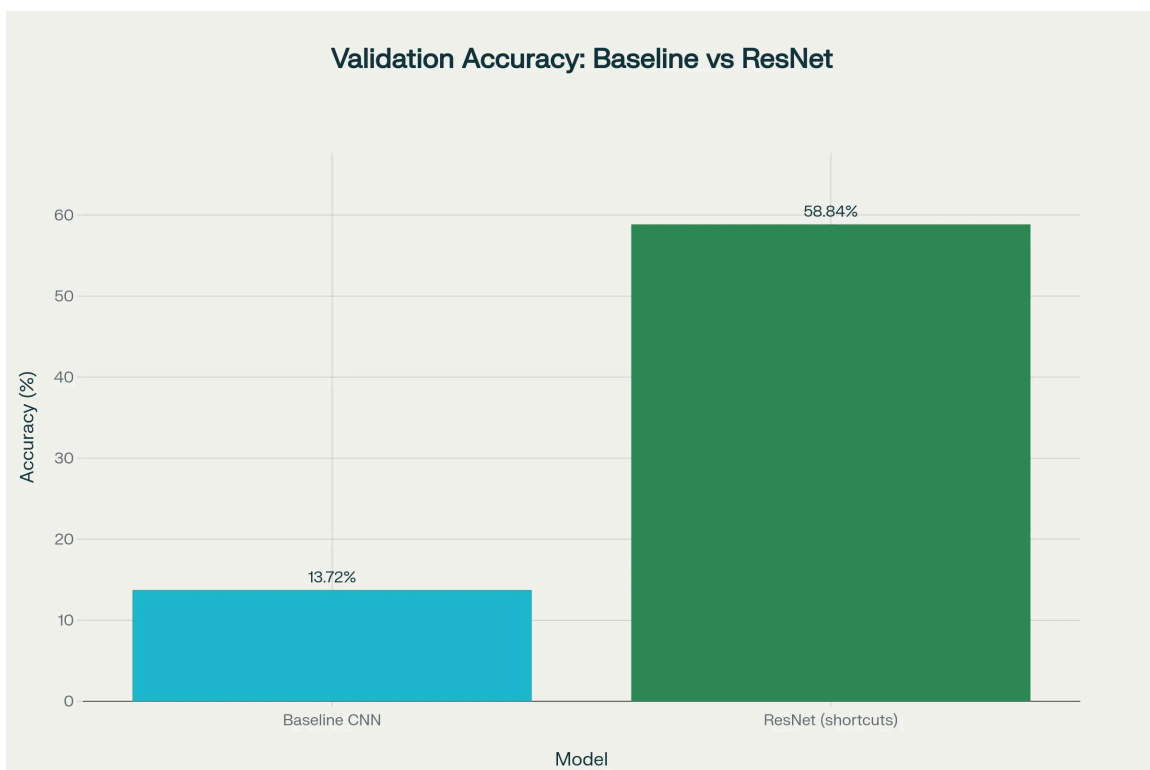


Figure 3: ResNet with skip connections achieves 4.3x better validation accuracy than the conventional baseline.

### 4.2. Limitations and Future Work

While conclusive, the study has limitations, including a modest dataset size and a final accuracy of 58.84% that indicates room for improvement. Future work could explore transfer learning with a pre-trained ResNet, deeper architectures (e.g., ResNet-50), advanced data augmentation, and attention mechanisms to further refine model capabilities.

## 5. Conclusion

---

This study unequivocally demonstrates that skip connections are a fundamental necessity for training deep CNNs. The stark contrast between the complete failure of the conventional CNN (13.72% accuracy) and the success of the ResNet (58.84% accuracy) is a powerful practical illustration of the vanishing gradient problem and its definitive solution. The results confirm that modern deep learning challenges demand modern architectures. The residual connection, elegant in its simplicity, transforms an intractable optimization problem into a solvable one, cementing its status as a cornerstone of contemporary neural network design.

## References

---

[1] Google Colab Tool

[2] Car Brand Logos Dataset (Kaggle): <https://www.kaggle.com/datasets/volkandl/car-brand-logos>

# Python Code

---

## Baseline CNN Model Definition

```
def build_baseline_model(input_shape, num_classes):
    model = Sequential([
        # Block 1
        Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=input_shape),
        BatchNormalization(),
        Conv2D(32, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),
        Dropout(0.25),
        # Block 2
        Conv2D(64, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        Conv2D(64, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),
        Dropout(0.25),
        # Block 3
        Conv2D(128, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        Conv2D(128, (3, 3), padding='same', activation='relu'),
        BatchNormalization(),
        MaxPooling2D(pool_size=(2, 2)),
        Dropout(0.25),
        # Classification Head
        Flatten(),
        Dense(256, activation='relu'),
        Dropout(0.5),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(num_classes, activation='softmax')
    ], name="BaselineCNN")
    return model
```



## ResNet Residual Block

```
def residual_block(x, filters, stride=1, conv_shortcut=False, name=None):
    shortcut = x
    if conv_shortcut:
        shortcut = layers.Conv2D(filters, 1, strides=stride, name=name + '_shortcut_conv', padding='same')(x)
        shortcut = layers.BatchNormalization(name=name + '_shortcut_bn')(shortcut)

    x = layers.Conv2D(filters, 3, strides=stride, padding='same', name=name + '_conv1')(x)
    x = layers.BatchNormalization(name=name + '_bn1')(x)
    x = layers.Activation('relu', name=name + '_relu1')(x)

    x = layers.Conv2D(filters, 3, padding='same', name=name + '_conv2')(x)
    x = layers.BatchNormalization(name=name + '_bn2')(x)

    x = layers.Add(name=name + '_add')([x, shortcut])
    x = layers.Activation('relu', name=name + '_relu2')(x)
    return x
```